

Sistemas Operativos

1º Parcial 1C2025 – TT – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría

1)

La técnica de *prevención* del deadlock se basa en establecer políticas del sistema operativo para asegurar que al menos una de las condiciones necesarias para que ocurra un deadlock (mutua exclusión, sin desalojo, retención y espera o espera circular) nunca se cumpla. Por ejemplo, se puede evitar la espera circular garantizando que los recursos se pidan siempre en un orden preestablecido.

Si comparamos ambas técnicas en términos de overhead vemos que la técnica de Prevención tiene un overhead mínimo ya que son políticas definidas a priori que luego no requieren de una ejecución activa para garantizarlas. En contraposición, la técnica de Evasión tiene un overhead alto ya que corre el algoritmo del banquero ante cada petición para decidir si los recursos son asignados o no.

En cuanto a la flexibilidad, vemos que Evasión tiene mayor flexibilidad que Prevención ya que no hay una definición previa de cómo van a ser asignados los recursos, siempre y cuando el sistema quede en estado seguro. La única limitación es la cantidad máxima de recursos que debe definir el proceso. En cambio, Prevención define ciertas políticas previamente que limitan la asignación de los mismos.

2)

Característica	Procesos	KLTs	ULTs
Multiprocesamiento	Sí (cada proceso puede correr en un core)	Sí (el kernel puede asignar hilos a distintos cores)	No (los hilos hermanos no podrán correr simultáneamente en cores distintos, solo puede suceder si pertenecen a KLTs distintos)
Overhead en creación	Alto (requiere estructura completa, creación del PCB, espacio de direcciones, etc.)	Medio (gestión en el kernel pero más liviano que un proceso, creación del TCB)	Bajo (se gestionan en espacio de usuario)
Seguridad / Aislamiento	Alta (memoria separada entre procesos)	Media (comparten espacio, pero gestionados por el kernel)	Baja (comparten memoria y el kernel no tiene control directo)

3)

Cuando finaliza el quantum (tiempo asignado al proceso en ejecución), el reloj del sistema genera una interrupción que es atendida por el sistema operativo.

- **Para ello:**

1. Se guarda el contexto del proceso actual (PC, registros, etc.) para atender la interrupción.
2. Como parte de la rutina de interrupción, el sistema operativo (planificador de corto plazo) pasará el proceso de ejecución a listo.
3. El planificador elige el próximo proceso a ejecutar en base al estado de la cola de listos.
4. Se carga el contexto del nuevo proceso seleccionado desde su PCB.

¿Puede ocurrir un cambio de proceso?

Sí, de hecho, esta es una de las principales situaciones donde se produce un cambio de proceso. Si hay otro proceso listo, el sistema lo seleccionará y desplazará al actual, que vuelve a la cola de listos.

4)

En un sistema que cuenta con un alto grado de uso de CPU, es conveniente utilizar una solución con bloqueo, como los semáforos.. Test and set es una solución con soporte de hardware (se

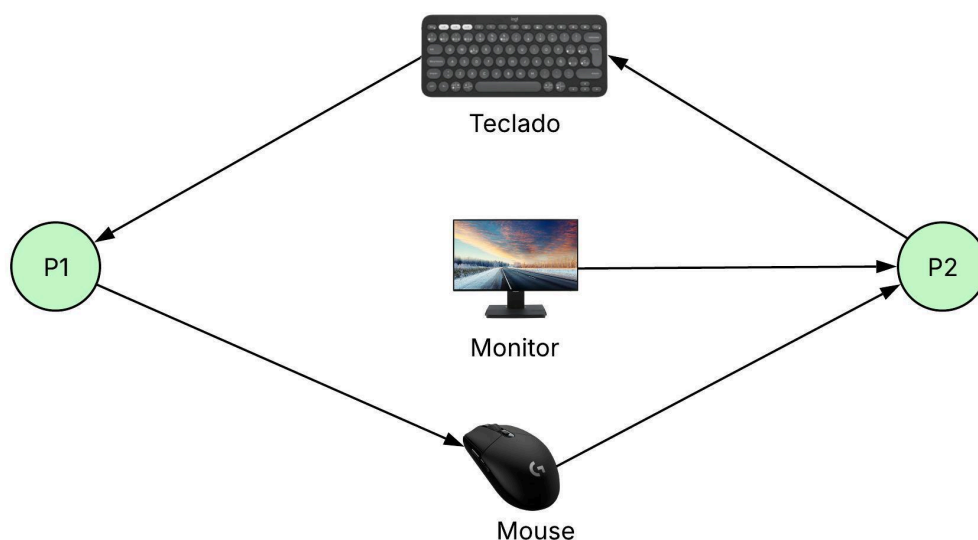
implementa utilizando una instrucción específica) que genera espera activa. En un sistema con condiciones de carrera muy frecuentes, esa espera activa generará aún más uso de la CPU, acrecentando el problema.

5)

- a. Falso. Si bien el planificador de largo plazo regula la cantidad de procesos admitidos al sistema (y por ende influye en el grado de multiprogramación), no es el único que interviene. El otro planificador que influye es el de mediano plazo, ya que al swapear procesos, el grado de multiprogramación disminuye. A su vez, cuando los procesos anteriormente suspendidos vuelven a memoria principal, el grado de multiprogramación aumenta.
- b. Verdadero. El algoritmo HRRN (*Highest Response Ratio Next*) calcula una prioridad que incluye el tiempo de espera. Esto hace que los procesos que esperan mucho tiempo aumenten su prioridad progresivamente, evitando así que queden postergados indefinidamente, siendo este último concepto la definición de inanición.

Práctica

- 1) No podemos garantizar que los procesos ejecuten sin inconvenientes, y esto se debe a varios factores. En un sistema donde los procesos ejecutan concurrentemente, podría suceder que el proceso 1 evalúe que todos los recursos a utilizar están disponibles. Sin embargo, antes que comience a utilizar los recursos, puede realizarse un cambio de proceso por el planificador de corto plazo, y hacer que el proceso 2 haga la misma evaluación. A partir de este instante, podemos ver que sufrimos de una condición de carrera ya que la utilización exitosa de los recursos o no, depende de la secuencia de ejecución de los procesos. Por otra parte, vemos que dependiendo del orden de petición de los recursos, se podría generar un deadlock en el sistema.



Las maneras posibles de solucionar este inconveniente son:

- Cambiando el orden de solicitud de los recursos, es decir, que la solicitud de recursos no sea invertida. Podemos definir que P1 pida [TECLADO, MOUSE]; mientras que P2 podría pedir [TECLADO, MOUSE, MONITOR], y todas sus variantes posibles.
- Otra forma es utilizando un semáforo mutex para la solicitud de los recursos. De esta manera, podemos evitar evaluar la condición del if, y dejaríamos de tener condición de carrera, ya que la función de este tipo de semáforo es garantizar que este fenómeno no suceda.

mutex_recursos = 1;	
Proceso 1	Proceso 2
<pre>while (true) { char* rec_a_usar = [TECLADO, MOUSE]; wait(mutex_recursos); usar_recurso(rec_a_usar[0]); usar_recurso(rec_a_usar[1]); signal(mutex_recursos); liberar_recursos(rec_a_usar); }</pre>	<pre>while (true) { char* rec_a_usar = [MOUSE, MONITOR, TECLADO]; wait(mutex_recursos); usar_recurso(rec_a_usar[0]); usar_recurso(rec_a_usar[1]); usar_recurso(rec_a_usar[2]); signal(mutex_recursos); liberar_recursos(rec_a_usar); }</pre>

2)

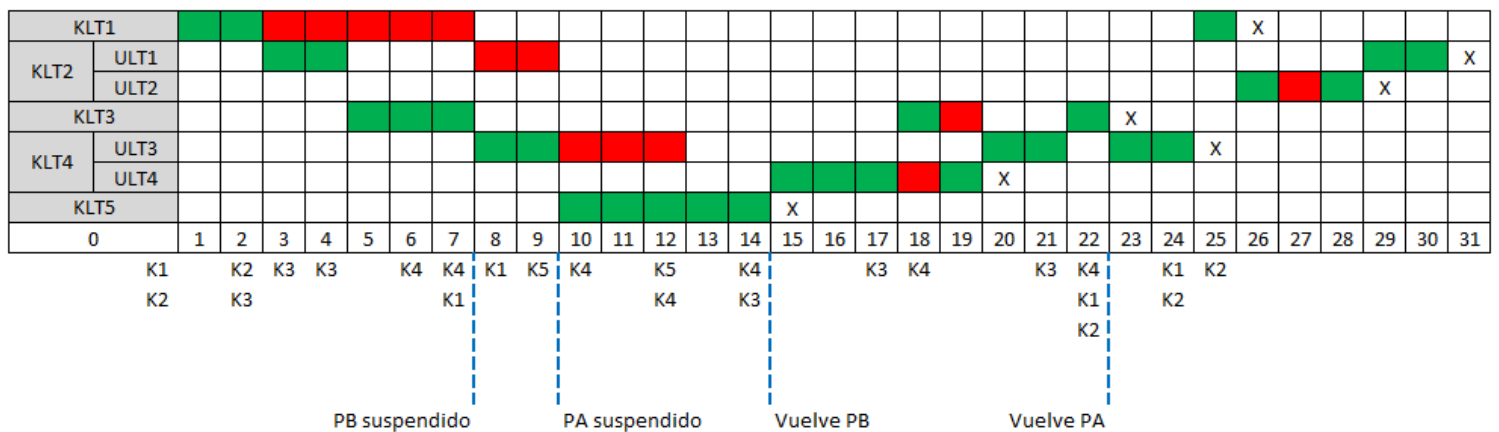
<pre>mutex_muestra = 1; reporte_publicado[3] = {0, 0, 0}; limite_repositorio = 20; muestra_disponible = 0;</pre>	
Analista (3 instancias)	Recolector de Muestras (N instancias)
<pre>while(1) { id_especialidad = getIdEspecialidad(); wait(reporte_publicado[id_especialidad]); revisar_reporte(); }</pre>	<pre>while(1) { muestra = recolectar_muestra(); wait(limite_repositorio); wait(mutex_muestra); depositar(muestra, repositorio_muestras); signal(mutex_muestra); signal(muestra_disponible); }</pre>
Bioquímica (1 instancia)	
<pre>while(1) { wait(muestra_disponible); wait(mutex_muestra); muestra = retirar_muestra(repositorio_muestras); signal(mutex_muestra); signal(limite_repositorio); procesar_muestra(muestra); publicar_reporte(); }</pre>	

```
signal(reporte_publicado[0]);
signal(reporte_publicado[1]);
signal(reporte_publicado[2]);
}
```

3)

a) Gantt

- Ráfagas de CPU representadas en verde.
- Ráfagas de IO representadas en rojo.



b) Syscalls

- $t = 0 \rightarrow$ creación de hilo KLT1, creación de hilo KLT2.
- $t = 2 \rightarrow$ creación de hilo KLT3, solicitud de IO de KLT1.
- $t = 4 \rightarrow$ solicitud de IO de KLT2 (ULT1).
- $t = 6 \rightarrow$ creación de hilo KLT4.
- $t = 9 \rightarrow$ creación de hilo KLT5, solicitud de IO de KLT4 (ULT3).
- $t = 14 \rightarrow$ finalización de KLT5.
- $t = 17 \rightarrow$ solicitud de IO de KLT4 (ULT4).
- $t = 18 \rightarrow$ solicitud de IO de KLT3.
- $t = 22 \rightarrow$ finalización de KLT3.
- $t = 24 \rightarrow$ finalización de KLT4.
- $t = 25 \rightarrow$ finalización de KLT1.
- $t = 26 \rightarrow$ solicitud de IO de KLT2 (ULT2).
- $t = 30 \rightarrow$ finalización de KLT2.